Lab 7: Fuzzing for Input Validation Bugs

Name: Sunay Hegde **SRN:** _____

Repository: SETAPESU26/L6_L7_58_restaurant_bill (LAB7) **Module:** restaurant_bill_parser.py
Tool: Hypothesis + pytest

Bugs found by fuzzing and how they were fixed

Function	Failing input (found by Hypothesis)	Crash	Fix
clean_dish_name	None	AttributeError: 'NoneType' has no attribute 'strip'	Return "" for any non-string input
parse_item_prices	'' (also "250,,320" in SS1)	ValueError: invalid literal for int(): ''	Parse each item separately with _to_int(); skip blank and non-numeric items
parse_item_prices	'-1'	No crash, but a negative price is returned (property fails)	Skip negative prices
parse_item_prices	None	AttributeError: 'NoneType' has no attribute 'split'	Return [] for any non-string input
split_amount	'' (no '/')	ValueError: not enough values to unpack	Return None unless there is exactly one '/'
split_amount	'/'	ValueError: invalid literal for int(): ''	Convert safely with _to_int(); return None if not a whole number
split_amount	'0/0' (earlier fuzz run)	ZeroDivisionError	Return None if people <= 0 (also rejects negative totals, e.g. '-1/1')
split_amount	None	AttributeError: 'NoneType' has no attribute 'split'	Return None for any non-string input

Each function now checks its input type first and never raises: invalid input returns a safe value ("", [] or None), and main() prints a clear message instead of a traceback.

SS1: python restaurant_bill_parser.py (buggy module)

```
PS D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7> python restaurant_bill_parser.py
==== Running clean_dish_name ====
Enter a dish name from the menu (e.g. *paneer tikka*): *paneer tikka*
Dish name: Paneer Tikka

==== Running parse_item_prices ====
Enter item prices separated by commas (e.g. 250,180,320): 250,180,320
Item prices: [250, 180, 320]

==== Running split_amount ====
Enter the bill and number of people as TOTAL/PEOPLE (e.g. 1200/4): 1200/4
Each person pays: 300
PS D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7> python restaurant_bill_parser.py
==== Running clean_dish_name ====
Enter a dish name from the menu (e.g. *paneer tikka*): #butter naan~
Dish name: Butter Naan

==== Running parse_item_prices ====
Enter item prices separated by commas (e.g. 250,180,320): 250,,320
Traceback (most recent call last):
  File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\restaurant_bill_parser.py", line 40, in <module>
    main()
    ~~~~^^
  File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\restaurant_bill_parser.py", line 33, in main
    print("Item prices:", parse_item_prices(value))
                          ~~~~~~~~~~~~~~~~~~^^^^^^^
  File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\restaurant_bill_parser.py", line 16, in parse_item_prices
    return [int(price) for price in text.split(',')]
            ~~~^^^^^^^
ValueError: invalid literal for int() with base 10: ''
PS D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7>
```

First run with valid input works. Second run crashes on "250,,320" with ValueError.

SS2: python -m pytest test_restaurant_bill_parser.py (buggy module) (part 1 of 3)

```
PS D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7> python -m pytest test_restaurant_bill_parser.py
============================= test session starts =============================
platform win32 -- Python 3.13.9, pytest-9.2.0, pluggy-1.7.0
rootdir: D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7
plugins: hypothesis-6.170.15, platformdirs-4.14.5
collected 3 items

test_restaurant_bill_parser.py FFF                                       [100%]

================================== FAILURES ===================================
_____________________________ test_clean_dish_name_no_crash _____________________________

s = None

    @settings(max_examples=300)
    @given(st.text() | st.none())
    def test_clean_dish_name_no_crash(s):
        try:
>           result = clean_dish_name(s)
                     ^^^^^^^^^^^^^^^^^^

test_restaurant_bill_parser.py:25:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

name = None

    def clean_dish_name(name):
        """
        Removes the chef's-special markers (*, #, ~) and title-cases the name.
        Assumes name is a non-empty string.
        """
>       name = name.strip()
               ^^^^^^^^^^
E       AttributeError: 'NoneType' object has no attribute 'strip'

restaurant_bill_parser.py:6: AttributeError

During handling of the above exception, another exception occurred:

    @settings(max_examples=300)
>   @given(st.text() | st.none())
                   ^^^

test_restaurant_bill_parser.py:22:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

s = None

    @settings(max_examples=300)
    @given(st.text() | st.none())
    def test_clean_dish_name_no_crash(s):
        try:
            result = clean_dish_name(s)
        except Exception as e:
>           assert False, f"clean_dish_name crashed with: {e!r}"
E           AssertionError: clean_dish_name crashed with: AttributeError("'NoneType' object has no attribute 'strip'")
E           assert False
E           Failing test case: test_clean_dish_name_no_crash(
E               s=None,
E           )

test_restaurant_bill_parser.py:27: AssertionError
```

SS2: python -m pytest test_restaurant_bill_parser.py (buggy module) (part 2 of 3)



```
_____________________ test_parse_item_prices_no_crash _____________________
  + Exception Group Traceback (most recent call last):
  |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\test_restaurant_bill_parser.py", line 33, in test_parse_item_prices_no_crash
  |     @given(st.text() | st.none() | price_like)
  |                    ^^^
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\test_restaurant_bill_parser.py", line 40, in test_parse_item_prices_no_crash
    |     assert all(isinstance(p, int) and p >= 0 for p in result)
    | AssertionError: assert False
    |  +  where False = all(<generator object test_parse_item_prices_no_crash.<locals>.<genexpr> at 0x0000014F8CEC3220>)
    | Failing test case: test_parse_item_prices_no_crash(
    |     s='-1',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\test_restaurant_bill_parser.py", line 36, in test_parse_item_prices_no_crash
    |     result = parse_item_prices(s)
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\restaurant_bill_parser.py", line 16, in parse_item_prices
    |     return [int(price) for price in text.split(',')]
    |             ~~~^^^^^^^
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\test_restaurant_bill_parser.py", line 38, in test_parse_item_prices_no_crash
    |     assert False, f"parse_item_prices crashed with: {e!r}"
    | AssertionError: parse_item_prices crashed with: ValueError("invalid literal for int() with base 10: ''")
    | assert False
    | Failing test case: test_parse_item_prices_no_crash(
    |     s='',
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\test_restaurant_bill_parser.py", line 36, in test_parse_item_prices_no_crash
    |     result = parse_item_prices(s)
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\restaurant_bill_parser.py", line 16, in parse_item_prices
    |     return [int(price) for price in text.split(',')]
    |                                     ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\test_restaurant_bill_parser.py", line 38, in test_parse_item_prices_no_crash
    |     assert False, f"parse_item_prices crashed with: {e!r}"
    | AssertionError: parse_item_prices crashed with: AttributeError("'NoneType' object has no attribute 'split'")
    | assert False
    | Failing test case: test_parse_item_prices_no_crash(
    |     s=None,
    | )
    +------------------------------------
_____________________ test_split_amount_no_crash ______________________
  + Exception Group Traceback (most recent call last):
  |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\test_restaurant_bill_parser.py", line 44, in test_split_amount_no_crash
  |     @given(st.text() | st.none() | split_like)
  |                    ^^^
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
```

SS2: python -m pytest test_restaurant_bill_parser.py (buggy module) (part 3 of 3)

```
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\test_restaurant_bill_parser.py", line 47, in test_split_amount_no_crash
    |     result = split_amount(s)
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\restaurant_bill_parser.py", line 24, in split_amount
    |     return int(total) // int(people)
    |            ~~~^^^^^^^
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\test_restaurant_bill_parser.py", line 49, in test_split_amount_no_crash
    |     assert False, f"split_amount crashed with: {e!r}"
    | AssertionError: split_amount crashed with: ValueError("invalid literal for int() with base 10: ''")
    | assert False
    | Failing test case: test_split_amount_no_crash(
    |     s='/',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\test_restaurant_bill_parser.py", line 47, in test_split_amount_no_crash
    |     result = split_amount(s)
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\restaurant_bill_parser.py", line 23, in split_amount
    |     total, people = text.split('/')
    |     ^^^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 2, got 1)
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\test_restaurant_bill_parser.py", line 49, in test_split_amount_no_crash
    |     assert False, f"split_amount crashed with: {e!r}"
    | AssertionError: split_amount crashed with: ValueError('not enough values to unpack (expected 2, got 1)')
    | assert False
    | Failing test case: test_split_amount_no_crash(
    |     s='',
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\test_restaurant_bill_parser.py", line 47, in test_split_amount_no_crash
    |     result = split_amount(s)
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\restaurant_bill_parser.py", line 23, in split_amount
    |     total, people = text.split('/')
    |                     ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7\test_restaurant_bill_parser.py", line 49, in test_split_amount_no_crash
    |     assert False, f"split_amount crashed with: {e!r}"
    | AssertionError: split_amount crashed with: AttributeError("'NoneType' object has no attribute 'split'")
    | assert False
    | Failing test case: test_split_amount_no_crash(
    |     s=None,
    | )
    +-----------------------------------
========================== short test summary info ==========================
FAILED test_restaurant_bill_parser.py::test_clean_dish_name_no_crash - Assert...
FAILED test_restaurant_bill_parser.py::test_parse_item_prices_no_crash - Exce...
FAILED test_restaurant_bill_parser.py::test_split_amount_no_crash - Exception...
============================ 3 failed in 2.84s ============================
PS D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7>
```

All 3 Hypothesis tests fail; Hypothesis shrinks each crash to the smallest failing input.

SS3: python restaurant_bill_parser.py (fixed module)

```
PS D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7> python restaurant_bill_parser.py
==== Running clean_dish_name ====
Enter a dish name from the menu (e.g. *paneer tikka*): *paneer tikka*
Dish name: Paneer Tikka

==== Running parse_item_prices ====
Enter item prices separated by commas (e.g. 250,180,320): 250,180,320
Item prices: [250, 180, 320]

==== Running split_amount ====
Enter the bill and number of people as TOTAL/PEOPLE (e.g. 1200/4): 1200/4
Each person pays: 300
PS D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7> python restaurant_bill_parser.py
==== Running clean_dish_name ====
Enter a dish name from the menu (e.g. *paneer tikka*): #butter naan~
Dish name: Butter Naan

==== Running parse_item_prices ====
Enter item prices separated by commas (e.g. 250,180,320): 250,,320
Item prices: [250, 320]

==== Running split_amount ====
Enter the bill and number of people as TOTAL/PEOPLE (e.g. 1200/4): 1200/0
Invalid input: use TOTAL/PEOPLE with whole numbers and at least 1 person
PS D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7> python restaurant_bill_parser.py
==== Running clean_dish_name ====
Enter a dish name from the menu (e.g. *paneer tikka*): ***
Dish name: (invalid dish name)

==== Running parse_item_prices ====
Enter item prices separated by commas (e.g. 250,180,320): abc,-50, 99 ,12.5
Item prices: [99]

==== Running split_amount ====
Enter the bill and number of people as TOTAL/PEOPLE (e.g. 1200/4): 900
Invalid input: use TOTAL/PEOPLE with whole numbers and at least 1 person
PS D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7>
```

The same bad inputs, plus more, are now handled without crashing.

SS4: python -m pytest test_restaurant_bill_parser.py (fixed module)

```
PS D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7> python -m pytest test_restaurant_bill_parser.py
============================= test session starts =============================
platform win32 -- Python 3.13.9, pytest-9.2.0, pluggy-1.7.0
rootdir: D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7
plugins: hypothesis-6.170.15, platformdirs-4.14.5
collected 3 items

test_restaurant_bill_parser.py ...                                       [100%]

============================== 3 passed in 2.77s ==============================
PS D:\PESU\Sem 5\SE\SE-Lab\Lab 6\L6_L7_58_restaurant_bill\LAB7>
```

All 3 Hypothesis tests pass.

Reflection

1. What kinds of bugs did fuzzing uncover that you might have missed with manual tests?

Hypothesis found crashes from inputs I would not normally type when testing by hand: **None** instead of a string, an **empty string**, an empty item between commas (**"250,,320"**), a split with **no '/'** or an empty side (**"/0"**), decimals, and **zero people** (ZeroDivisionError). My manual tests used well-formed examples like "1200/4", so they all passed. Hypothesis also shrinks each failure to the smallest input that still fails (e.g. **s=None**, **s=''**), which made the cause obvious.

2. How did you fix the bugs, and how does defensive programming make the code more robust?

Each function now validates its input before using it: it checks the type, converts numbers through a helper that catches ValueError, checks the shape of the input (exactly one '/'), and checks ranges (people > 0, no negative amounts). Invalid input returns a safe value ("", [] or None) instead of raising, and main() prints a clear message. Defensive programming means the program never trusts what a user typed, so one bad value cannot crash the whole bill calculation, and the error messages tell the user what to fix.

3. What are the limitations of fuzz testing, and how would you use it in real projects?

Fuzzing mainly proves that code does not crash; it cannot tell whether an answer is correct unless you also write properties for it (I added checks such as "prices are non-negative ints" and "no * # ~ markers remain"). It is random, so it can miss rare inputs, and it needs good strategies to reach deep code paths. In real projects I would run Hypothesis tests in CI next to normal unit tests, focus them on code that parses user or external input, and turn every failure Hypothesis finds into a permanent regression test.